INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 5

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica es fa una introducció al simulador i8085, juntament amb la realització de codis en aquest simulador. Es faran comparacions amb el simulador que s'ha utilitzat fins ara, que és el simulador Ripes. També es treballaran les subrutines en els dos simuladors.

OBJECTIUS

Com a objectius de la pràctica es té familiaritzar-se amb les instruccions del simulador i8085 i les funcions d'aquest. A més a més, també es té com a objectiu aprendre els diferents modes d'adreçament d'aquest simulador, junt amb aprendre a utilitzar les subrutines en els dos simuladors, el Ripes i el i8085.

EXERCICI 1

Feu de nou els problemes plantejats a la pràctica 3 (Fibonacci i algorisme d'Euclides per calcular el m.c.d.) però ara utilitzant subrutines.

A continuació, es mostra el codi que es va utilitzar per resoldre el problema de la pràctica 3 de l'algorisme d'Euclides:

```
.data
        a: .word 252
        b: .word 105
        resultat: .word 0
.text
        la a0, a
        lw a1, 0(a0)
        lw a2, 4(a0)
# Bucle que realitza l'algorisme d'Euclides
loop:
        beq a1, a2, end         # condició del  while, és a dir si a1 == a2, es salta a end
        bgt a1, a2, cert        # si es compleix que a1 > a2
        j fals                  # salt incondicional si no es compleix que a1 > a2, és a dir
                                  vol dir que a1 < 2
cert:
        sub a1, a1, a2          # a1 = a1-a2
```

```
        j loop                  # Salt incondicional enrere per tornar a fer el loop
fals:
        sub a2, a2, a1          # a2 = a2-a1
        j loop                  # Salt incondicional enrere per tornar a fer el loop
end:
        sw a1, 8(a0)            # Es guarda el valor de a1 en la posició de memòria [a0] +
                                  8
```

Ara, es demana utilitzar subrutines per tal de contestar a la qüestió que es demana.
Per tant, el codi és el següent:

```
.data
        a: .word 252
        b: .word 105
        resultat: .word 0
.text
        la a0, a
        lw a1, 0(a0)
        lw a2, 4(a0)
        call subrutina
         sw a3, 8(a0)           # Es guarda el valor de a1 en la posició de memòria [a0]+8
        j end2                  # Salt incondicional per evitar tornar a fer la subrutina.

.text
subrutina:
        addi sp, sp, -8
        sw a1, 0(sp)
        sw a2, 4(sp)
# Bucle que realitza l'algorisme d'Euclides
loop:
        beq a1, a2, end         # condició del  while, és a dir si a1 == a2, es salta a end
        bgt a1, a2, cert        # si es compleix que a1 > a2
        j fals                  # salt incondicional si no es compleix que a1 > a2, és a dir
                                # vol dir que a1 < 2
cert:
        sub a1, a1, a2          # a1 = a1-a2
        j loop                  # Salt incondicional enrere per tornar a fer el loop
fals:
        sub a2, a2, a1     # a2 = a2-a1
        j loop                  # Salt incondicional enrere per tornar a fer el loop
end:
        mv a3, a1
        lw a1, 0(sp)
        lw a2, 4(sp)
        ret
```

Per evitar el while true proposat en el codi de factorial, es fa un salt incondicional a aquesta posició de memòria

```
end2:
        nop
```

Respecte al codi de Fibonacci, tenim que el codi que es va utilitzar a la pràctica 3 va ser el següent:

```
.data
comptador: .word 10
resultat: .word 0
.text
la a0, comptador            # S'assigna a a0 l'adreça de comptador
lw a0, 0(a0)                # Carreguem el valor de 0(a0) en a0
addi a1, zero, 0            # Assignem a a1 el valor 0 (F_0)
addi a2, zero, 1            # Assignem a a2 el valor 1 (F_1)

# És un loop que funciona mentre a0 > 0
loop:
   blez a0, end             # Condició per tal de continuar el loop
   add a3,a1, a2            # Definició de Fibonacci
   addi a1, a2, 0           # Serveix per guardar valor a2 en a1
   addi a2, a3, 0           # Serveix per guardar valor a3 en a2
   addi a0, a0, -1          # Es decrementa el valor de a0 en 1
   j loop                   # Salt incondicional per tal de repetir el loop
end:
   la a0, resultat          # S'assigna a a0 l'adreça de resultat
   sw a1, 0(a0)             # Es guarda el valor de a1 en 0(a0)
```

Per tant, el codi de Fibonacci utilitzant subrutines és el següent:

```
.data
```

```
comptador: .word 10
resultat: .word 0
.text
la a0, comptador            # S'assigna a a0 l'adreça de comptador
lw a0, 0(a0)                # Carreguem el valor de 0(a0) en a0
addi a1, zero, 0            # Assignem a a1 el valor 0 (F_0)
addi a2, zero, 1            # Assignem a a2 el valor 1 (F_1)
call subrutina              # S'assigna a a0 l'adreça de resultat
sw a3, 4(a0)                # Es guarda el valor de a1 en 0(a0)
j end2
.text
subrutina:
   addi sp,sp, -12
   sw a1, 0(sp)
   sw a2, 4(sp)
   sw a0, 8(sp)
# És un loop que funciona mentre a0 > 0
loop:
   blez a0, end             # Condició per tal de continuar el loop
   add a3,a1, a2            # Definició de Fibonacci
   addi a1, a2, 0           # Serveix per guardar valor a2 en a1
   addi a2, a3, 0           # Serveix per guardar valor a3 en a2
   addi a0, a0, -1          # Es decrementa el valor de a0 en 1
   j loop                   # Salt incondicional per tal de repetir el loop
end:
   mv a3, a1
   lw a1, 0(sp)
   lw a2, 4(sp)
   lw a0, 8(sp)

# Per evitar el while true proposat en el codi de factorial, es fa un salt incondicional a
aquesta posició de memòria

end2:
   nop
```

EXERCICI 2

Escriviu dos programes diferents per sumar dues matrius de 5x1 i desar el resultat en una tercera, podeu fer servir adreçament indirecte, per parella de registres o per registres. Penseu en dues hipòtesis, primera, les matrius d'entrada s'han de conservar, és adir el resultat s'emmagatzema sobre mat3 i segona, desem el resultat sobre una de les dues matrius d'entrada, mat3 és irrellevant. El programa hauria d'utilitzar un comptador, instruccions d'increment, comparació i salt condicional. Feu la suma de números hexadecimals i sense tenir en compte números que pugui donar

overflow. Compareu el resultat de tots dos programes amb el programa fet en Ripes. Quin triga més? Considereu diferents tipus de processador RISC-V.

Veiem primer el codi en el cas de guardar els resultats en la matriu 2, on la matriu 3 és irrellevant:

```
.define
num 5
.data 00h
mat1: db 1,2,3,4,5
mat2: db 6,7,8,9,0
mat3: db 0,0,0,0,0
.org 100h
      lxi b, mat1
      lxi h, mat2
      mvi d, num
      loop:
            dcr d
            jm final
            ldax b
            mov e, M
            add e
            mov M, a
            inx h
            inx b
            jmp loop
      final:
hlt
```

El que està fent el codi és, a partir d'un comptador que és *num,* el programa va sumant els elements de cada matriu i es van guardant en la matriu 2, i quan acaba la suma dels elements de la posició n de cada matriu, s'incrementa la posició de memòria en 1, per tal de sumar els següents elements. Tot això es fa en un bucle, que té com a nom d'etiqueta *loop.* Quan aquest comptador, després d'haver estat decrementat en cada iteració, arriba a ser negatiu, aleshores es salta a *final*, per tal de sortir del loop i finalitzar el programa.

A continuació, tenim el segon codi, on els resultat es guarden en la matriu 3:

```
.define
num 5
.data 00h
mat1: db 1,2,3,4,5
mat2: db 6,7,8,9,0
mat3: db 0,0,0,0,0
.org 100h
      lxi b, mat1
      lxi h, mat3
      mvi d, num
      loop:
            dcr d
            jm next
            ldax b
            mov e, M
```

```
            add e
            mov M, a
            inx h
            inx b
            jmp loop
      next:
            lxi b, mat2
            lxi h, mat3
            mvi d, num
      loop2:
            dcr d
            jm final
            ldax b
            mov e, M
            add e
            mov M, a
            inx h
            inx b
            jmp loop2
      final:
hlt
```

Tal i com es pot observar, si tenim en compte que hem de guardar el resultat en la matriu 3, té el mateix funcionament el codi 2 que el codi 1, amb la diferència que es fan dos bucles. El primer bucle es per passar els valors de la matriu 1 a la matriu 3, ja que la matriu 3 és plena de zeros, i per tant és com copiar matrius. Després, en el segon bucle es fan la suma dels elements de la matriu 2 amb la matriu 3, de forma que la matriu 1 i la matriu 2 queden intactes i la matriu 3 té els resultats de la suma de les dues matrius anteriors.

Pregunta 1

Compareu els modes d'adreçament del simulador i8085 amb el simulador Ripes.

Tal i com es va veure en el simulador Ripes, aquest en tenia diferents modes d'adreçament, com eren l'implícit, el directe, el relatiu, l'immediat i l'indexat. En canvi al simulador i8085, trobem que tant l'implícit (que el podem veure amb la instrucció add e, ja que s'està sumant a l'acumulador i guardant a aquest), com el directe i el relatiu també estan (tal i com es pot comprovar amb la instrucció jm final). No obstant, a diferència de Ripes, veiem que si que inclou el mode d'adreçament indirecte, tal i com es pot veure amb la instrucció mov e, M. Però malgrat això, el mode d'adreçament indexat no està disponible al i8085, cosa que si fos disponible ens seria de molta utilitat de cara a la realització dels codis anteriors, ja que seria molt més fàcil moure's pels elements de les matrius.

Veiem ara el nombre de cicles i execucions dels nostres codis en el simulador i8085, per tal de poder comparar-los després amb el nombre de cicles en el simulador de Ripes:

Primer codi		
Instrucció	Cicles	Execucions
LXI rp, da16	10	2
MVI reg, dat	7	1
JM label	7/10	4/1
MOV reg, M/MOV M, reg	7	10
LDAX rp	7	5
ADD reg	4	5
STAX rp	7	5
INX rp	6	10
DCR reg	4	6
JMP label	10	5
HTL	4	1
Total	363	55

Segon codi		
Instrucció	Cicles	Execucions
LXI rp, da16	10	4
MVI reg, dat	7	2
JM label	7/10	8/2
MOV reg, M/MOV M, reg	7	20
LDAX rp	7	10
ADD reg	4	10
STAX rp	7	10
INX rp	6	20
DCR reg	4	12
JMP label	10	10
HTL	4	1
Total	722	109

Veiem que evidentment, el segon codi, en el que hem de guardar els resultats en la matriu 3, té més cicles que el primer. Si fem la divisió del nombre de cicles entre el nombre d'execucions, veiem que en el cas del codi 1 tenim que dóna 6,6 cicles per execució, mentre que en el cas del segon codi tenim que dóna 6,62 cicles per execució. Per tant veiem que per cada execució, el codi 2 té un promig de cicles més elevat que el codi 1.

A continuació, veiem el com seria el codi 1 en el simulador de Ripes:

.data

```
   mat1: .word 1,2,3,4,5
   mat2: .word 6,7,8,9,0
   mat3: .word 0,0,0,0,0

.text
   main:
      la a0, mat1 # Direcció de mat1
      la a4, mat2 # Direcció de mat2

# Tenint en compte que inicialment 0(a0) representa el primer element de mat1
# i que 20(a0) el primer element de mat2, ens mourem per les matriu mantenint
aquesta distància entre posicions.

   loop:
      lw a1, 0(a0) # Carreguem el valor de la posició n de mat1
      lw a2, 20(a0) # Carreguem el valor de la posició n de mat2
      add a2, a2, a1 # Fem la suma dels continguts d'aquestes posicions
      sw a2, 20(a0) # Guardem el resultat de la suma
      addi a0,a0,4 # Incrementem a0 per tal d'anar a la següent posició de memòria de
      beq a0, a4, end # Si s'ha arribat a la primera posició de mat1, sortim del bucle
      j loop
   end:
      nop
```

Veiem que en el cas d'utilitzar el Pipeline de 5 etapes, tenim que triga 58 cicles en realitzar el programa, mentre que amb el Single-Cycle Processor, es triguen 39 cicles. Observem que el programa té 10 instruccions, per tant fent un càlcul ràpid veiem que per cada instrucció, en el cas del Single-Cycle es fan 3,9 cicles, mentre que amb el Pipeline de 5 etapes 5,8.

A continuació, veiem com seria el codi 2 en el simulador de Ripes:

```
.data
   mat1: .word 1,2,3,4,5
   mat2: .word 6,7,8,9,0
   mat3: .word 0,0,0,0,0

.text
   main:
      la a0, mat1 # Direcció de mat1
      la a4, mat2 # Direcció de mat2

# Tenint en compte que inicialment 0(a0) representa el primer element de mat1
# i que 20(a0) el primer element de mat2, ens mourem per les matriu mantenint
aquesta distància entre posicions.
# Per altra banda, el primer element de mat3 està en 40(a0)
   loop:
```

```
    lw a1, 0(a0) # Carreguem el valor de la posició n de mat1
    lw a2, 20(a0) # Carreguem el valor de la posició n de mat2
    add a3, a2, a1 # Fem la suma dels continguts d'aquestes posicions
    sw a3, 40(a0) # Guardem el resultat de la suma
    addi a0,a0,4 # Incrementem a0 per tal d'anar a la següent posició de memòria de
    beq a0, a4, end # Si s'ha arribat a la primera posició de mat1, sortim del bucle
    j loop
end:
    nop
```

Observem que el codi és pràcticament idèntic i amb el mateix nombre d'instruccions, només fent el canvi en la instrucció sw a3, 40(a0) per guardar els resultats en la mat3 i la instrucció add a3, a2, a1, en la que es fa ús del registre a3. Per tant, el nombre de cicles tant al Pipeline de 5 etapes com al Single-Cycle és el mateix que en el codi anterior, i per tant el càlcul del nombre de cicles per instrucció és el mateix.

Pregunta 2

Quants cicles de rellotge triga en executar-se una instrucció aritmètic – lògica qualsevol, en i8085? Compara el resultat amb el processador RISC-V

Veiem que en aquest cas tenim dos tipus d'operacions aritmètic-lògiques. La diferència entre aquestes dues és si s'utilitzen registres o s'ha d'accedir a memòria.

Tal i com passava al simulador Ripes, les instruccions que havien d'accedir a memòria trigaven més cicles que les que treballaven amb registres. Per tant, veiem que en el simulador i8085 passarà una cosa similar, ja que si treballem amb registres només es necessitaran 4 cicles, mentre que si hem d'accedir a memòria, es necessitaran 7 cicles.

Pregunta 3

Pujeu el vostre codi i marqueu quina és la instrucció del vostre programa que triga més cicles en executar-se

Veiem que en el cas dels nostres codis, com es repeteixen les instruccions, la resposat val pels dos codis. S'observa que, d'acord amb la taula que s'ha donat prèviament, tenim que la instrucció LXI rp, da16 triga 10 cicles, a més a més de les instruccions JMP label i la instrucció JM label, que triguen 10 cicles les dues; en el cas de la segona trigarà això en el cas de que no es compleixi la instrucció.

EXERCICI 3

Fent ús del programa anterior, realitzeu el mateix exercici amb subrutines amb tots dos simuladors. Feu ús dels coneixements ja desenvolupats amb RISC-V per implementar el codi en el processador i8085. Un cop fet això, feu una subrutina que codifiqui una zona de memòria. En el cas del i8085, la zona de memòria s'indicarà posant al registre doble HL l'adreça de començament de la zona de dades a codificar. Aquestes dades es consideren com els paràmetres d'entrada de la subrutina. La codificació es farà mitjançant una XOR entre cada byte de la zona de dades i la clau. Els valors dels registres no han de quedar afectats per la crida a la subrutina. Feu un programa que faci us d'aquesta subrutina per provar-la amb diferents combinacions de valors de la clau i les dades per codificar.

En primer lloc, es dona el codi fet en i8085, utilitzant subrutines:

```
.define
num 5
.data 00h
mat1: db 1,2,3,4,5
mat2: db 6,7,8,9,0
mat3: db 0,0,0,0,0
valorpila: dw 300h
tmp: db 0
.org 100h
      lxi h, mat3
      shld tmp
      lxi b, mat1
      lxi h, mat2
      mvi d, num
      loop:
            dcr d
            jm final
            ldax b
            mov e, M
            call subrutina
            inx h
            inx b
            jmp loop
      final:
hlt
.org 200h
subrutina:
      push h
      push b
      push d
      lhld tmp
      add e
      mov M, a
      inx h
      shld tmp
      pop d
      pop b
      pop h
ret
```

A continuació, es dona el codi fet amb subrutina en el simulador Ripes:

```
.data
   mat1: .word 1,2,3,4,5
   mat2: .word 6,7,8,9,0
   mat3: .word 0,0,0,0,0

.text
   main:
      la a0, mat1 # Direcció de mat1
      la a4, mat2 # Direcció de mat2

# Tenint en compte que inicialment 0(a0) representa el primer element de mat1
# i que 20(a0) el primer element de mat2, ens mourem per les matriu mantenint
# aquesta distància entre posicions.
# Per altra banda, el primer element de mat3 està en 40(a0)
   loop:
      lw a1, 0(a0) # Carreguem el valor de la posició n de mat1
      lw a2, 20(a0) # Carreguem el valor de la posició n de mat2
      call subrutina # Es guarda el valor del PC actual
      sw a3, 40(a0) # Guardem el resultat de la suma
      addi a0,a0,4 # Incrementem a0 per tal d'anar a la següent posició de memòria de
      beq a0, a4, end # Si s'ha arribat a la primera posició de mat1, sortim del bucle
      j loop
.text
subrutina:
   addi sp, sp, -8 # Es reserva espai a la pila
   sw a1, 0(sp) # Es guardan els valors dels registres a la pila per tal de no perdre'ls
   sw a2, 4(sp)
   add a2, a2, a1
   mv a3, a2     # Es mou el contingut del registre a2 al registre a3
   lw a2, 4(sp)  # Es tornen a carregar els valors que s'havien guardat a la pila
   lw a1, 0(sp)
   ret           # Tornem a tenir el valor del PC actual i retornem on estava el programa
end:
   nop
```

A continuació, es dona el codi en el que es codifica una zona de memòria amb XOR:

```
.define
      num 5
.data 00h
      mat1: db 1,2,3,4,5
      mat2: db 6,7,8,9,0
.org 100h
      lxi h, mat1
      lxi d, mat2
      mvi b, FFh
      mvi c, num

loop:
      dcr c
      jm final
      ldax d
      add M
      call codifica
      inx h
      inx d
      jmp loop

final:
hlt
.org 200h
codifica:
      push h
      xra b
      mov M, a
      pop h
ret
```

Observem que aquest codi és bastant semblant al que havíem fet abans en l'exercici 2. En aquest cas, ens permet codificar i decodificar la suma de les matrius, ja que XOR és una funció reversible i aquesta es veu manifestada en la instrucció *xra*. Per exemple, si prenem mat1: 1,2,3,4,5 i mat2:6,7,8,9,0 , la instrucció mov M, a ens dóna com a resultat F8, F6, F4, F2, FA en hexadecimal. Si prenem mat1: F8, F6, F4, F2, FA i mat2:0,0,0,0,0 aquesta mateixa instrucció ens dóna com a resultat 7, 9, B, D, 5 en hexadecimal, és a dir, la suma de mat1 i mat2 inicials.

Pregunta 4

Quina instrucció fem servir per assignar la posició inicial al registre SP? Raona la resposta

Fem servir la instrucció *call* per donar pas a la subrutina, a la cual el registre SP deixa de tenir els bytes en 0000, ja que en el moment en el que es fa el *push* a la subrutina, el SP té FFFE com a posició, i si es fa un altre *push*, aleshores serà FFFD. Per tant, en el moment en que es fa la instrucció *call*, se li està assignant una posició inicial.

Pregunta 5

Quina es la instrucció utilitzada per guardar el PC en la pila quan treballem amb subrutines? I per recuperar de nou el valor del PC?

La instrucció *call* guarda en una pila el valor actual del PC, de forma que queda emmagatzemat, i quan s'executa la instrucció *ret*, recuperem el valor del PC i podem tornar al fluxe normal del programa.

CONCLUSIONS

Com a conclusions d'aquesta pràctica tenim que s'ha pogut aprendre a utilitzar per primera vegada el simulador i8085, on s'ha après a realitzar codi i a observar com va canviant el contingut de la memòria a mesura que es van realitzant les instruccions.

Per altra banda, s'ha pogut comprovar els diferents modes d'adreçament que hi ha en el simulador i8085 i comparar-los amb els que ofereix el simulador Ripes.

A més a més, s'ha fet una introducció a les subrutines i com fer-ne ús en els dos simuladors de l'assignatura.